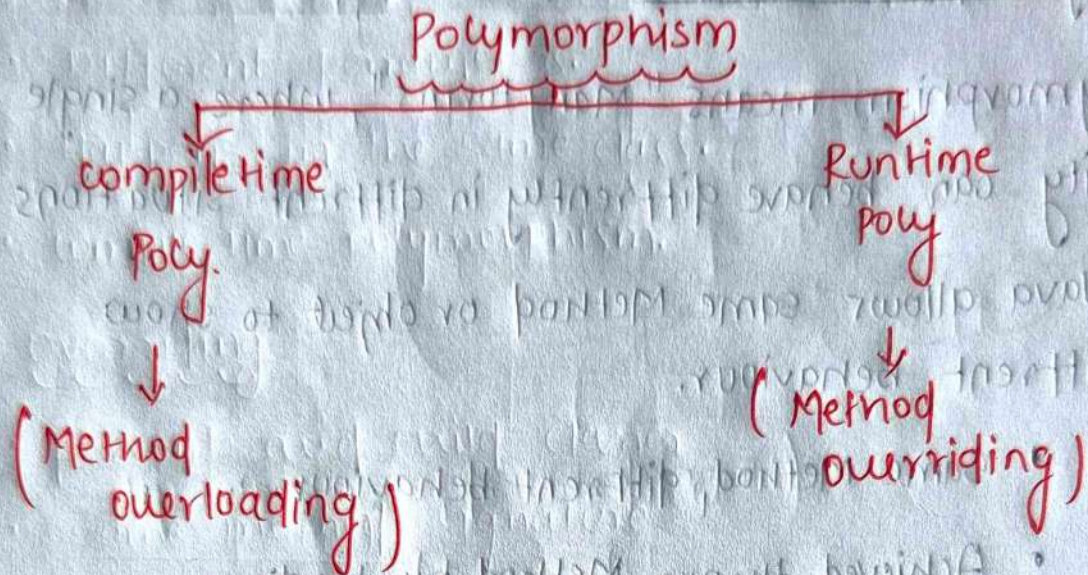


compile time → Before running checked By compiler
Runtime → while running executed by JVM/CPU.

→ Types of Polymorphism

(Mainly of two types)



→ compile time (Method overloading) → Achieved when multiple Methods have same name but different parameter. The Method call is resolved at ~~runtime~~ compile time based on ~~object~~.

→ Runtime (Method overriding) → (Also called as dynamic Method dispatch)

→ Achieved when a subclass provides a specific implementation of a Method already defined in its superclass. The Method call is resolved at runtime based on object.

1) Class vs Object

- A class is a blueprint of the object.
- An object is the real instance created from the class.

2) constructor

- constructor has the same name as the class.
- It has no return type (Not even void)
- It is called automatically when object is created.

3) this keyword

- Refers to the current object.
- used to distinguish instance variable from local variable

// →

```
class Student {
```

```
    String name;
```

```
    Student (String name) {
```

```
        this.name = name;
```

```
    }
```

4) Static vs Instance.

```
Static int count; // Shared by all objects
```

```
int age; // separate copy for each object.
```


→ 5) Method overloading vs overriding.

→ Overloading

- Same Method Name
- Different parameters.
- Happens in the same class.
- compile time polymorphism.

→ Overriding

- parent and child classes
- same method signature
- Run time polymorphism.

→ 6) Super keyword.

- used to access parent class Members.

→ 7) final

- final variable → cannot be reassigned
- final Method → cannot be overridden
- final class → cannot be inherited.

→ 8) upcasting and downcasting.

Animal a = new Dog (); // upcasting

Dog d = (Dog) a; // down casting.

9) why constructors are Not inherited

→ constructor belongs to object creation and every class has there own object.

10) Difference Between object Reference and object.

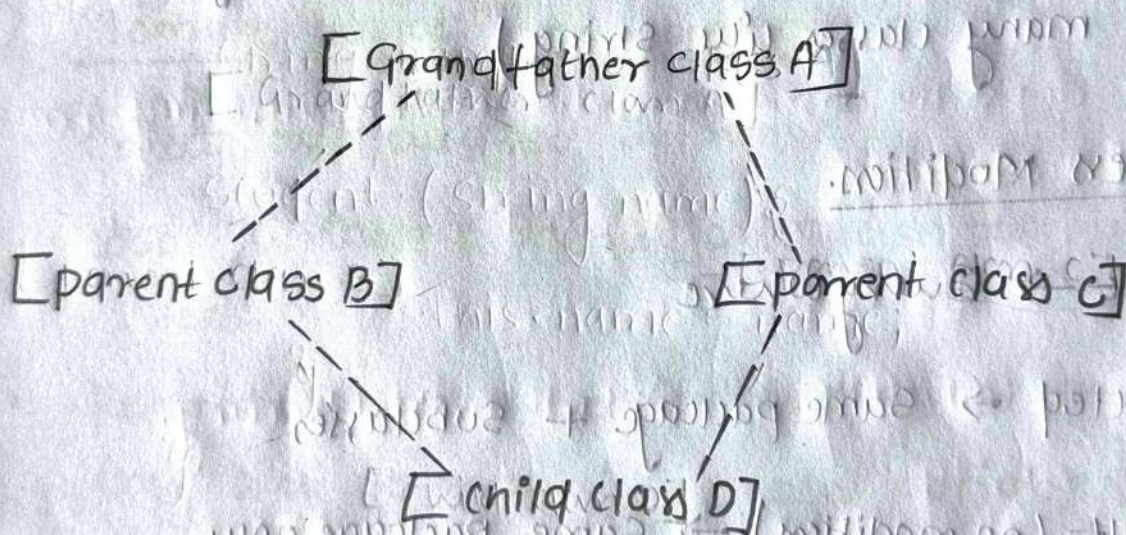
Student s;

// Reference

s = new Student(); // object created.

11) why java doesn't support Multiple inheritance with classes.

→ to avoid diamond problem, this happens when a subclass inherits from two parent classes that both modify the same Method from a grand father class.



→ if class A has a Method, and Both B & C rewrite that Method differently, class D does Not know which parent Method to execute.

→ 12) INTERFACE

ABSTRACT CLASSES

1) Multiple inheritance

single inheritance

2) No constructor

can have constructor

3) used for a contract

used for partial implementation

→ 13) == vs. equals()

`==` // compare references

`.equals()` // compare content (for many classes many classes use string)

→ 14) Access Modifiers

- Public → Everywhere
- Protected → Same package + subclasses
- Default (no modifiers) → Same package only
- Private → Same class only.

16) why polymorphism

to write flexible and reusable code.

→ `Animal a = new dog();`

→ `a.sound();`

→ the Method executed depends on the actual object (dog), not the reference type (Animal).

17) Object class

→ Every java class implicitly extends object, so methods like `toString()`, `equals()`, `hashCode()`, and `getClass()` are available to all concept object.

18) Memory basics (very common)

- Object are created in heap.
- local variable & Method calls use the Stack
- References are stored according to where they are declared (for local variable, on the stack).

19) can we create object of an abstract class?

No. But you can create a reference.

`Animal a = new dog();`

Remember the problem each oop

solves

concept

Problem it solves

class

Defines a blueprint for creating similar objects

object

Represents a real world entity memory.

constructor

Ensures an object is initialized when created.

Encapsulation

prevents invalid data by providing controlled access.

inheritance

Eliminate code duplication and promotes code reuse.

Polymorphism

Allows the same interface to behave differently depending on the object.

Abstraction

Hides implementation details and exposes only essential behaviour

2> compile time vs runtime (Most important)

• compile time

java checks the reference typ.

Vehicle v = new car();
v.start();
Reference type → actual object

• Run time

Java checks actual object.

Vehicle v = new car();

The actual object is car

if car overrides start(), java executes

car.start();

important Note

compile time = Reference type

Run time = Actual object

3> upcasting and down casting.

• upcasting

Vehicle v = new car();

• Happens automatically

• Safe

• Access only parent methods through the reference

• Down casting.

`((Car)v).openSunroof();`

• Explicit cast

• used to access child specific methods.

• safe only if the object is actually a car.

4) Encapsulation

Encapsulation mean providing controlled and validated access to data.

Private + validation = Good Encapsulation

5) constructor

• Same name as the class

• No return type

• called automatically when an object is created

• used to initialize objects.

6) Inheritance

inheritance solves

• code duplication

• code reuse

• Easier maintainance

• Better scalability

• is-A relationship

Ex → Dog is a Animal

7) Method overloading.

conditions

- Same Method Name
- Same parameters
- Parent-child relationship
- Runtime polymorphism

Method are overridden. variables are not.

8) Java is Not purely object-oriented.

correct interview answer.

- primitive data types (int, char, boolean, etc),
- Static Members can be used without creating an object.

Don't say →

Because Java doesn't support multiple inheritance.

• Mistakes I did

1) compile time → Method overloading

Runtime → Method overriding

2) Downcasting only changes how the reference is treated. the object already has those Methods.

→ 10 Rules I must Never forget.

compile time = Reference type

• Runtime = Actual objects

• overloading = compile time

• overriding = Runtime

• Inheritance solves code duplication

• Encapsulation = controlled & validated Access

• Every object gets its own copy of instance variables

• constructors initialize object automatically

• java is purely oop because primitive types are not objects

• down casting doesn't create Method - it allows access to child specific Methods when the object is actually of that child type.

up casting, down casting, casting, polymorphism

with polymorphism, we can have overriding

?? (if b/w)

up casting only changes how the reference

variable is stored in memory, but the object itself is not changed

down casting is the opposite of up casting

private → because private method cannot be overridden

final → final method cannot be overridden

static → static method belongs to class, cannot be overridden

Abstraction (oop) Abstract → existing in thought or as an idea but not having a physical or concrete existence

Abstraction is a process of hiding implementation details & only showing essential features of an object & also supports loose coupling through abstract classes & interfaces

loose coupling means two classes depend as little as possible on each other. Instead depending on a concrete class

abstract Method → An abstract Method is declared without any implementation (i.e. body)
→ sub classes must provide the implementation

→ declared using the abstract keyword

→ A class containing one or more abstract Method must be declared abstract

→ Abstract Method cannot use the following Modifiers (final, static, private, Synchronised, native, Strictfp)

→ final → because the final Method cannot be overridden but an abstract Method must be overridden

→ private → because private Method cannot be inherited so they cannot be overridden

→ static → static Method belongs to class, cannot be overridden

Abstract class.

→ the abstract class contains abstract Methods, then the class itself must be declared abstract as in

```
Public abstract class phoneModel {
```

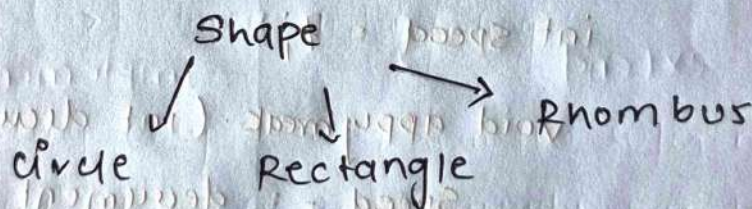
```
    abstract void switch off () {};
```

```
}
```

→ When an abstract class is subclassed, the subclass must provide implementation for all the Method in the parent class or declared abstract

(An abstract class May contain)

- Abstract Method (Method without body)
- concrete Method (Method with implementation)
- constructors
- instance variables
- static & final Methods



→ it is possible to create a reference of an abstract class

→ it is Not possible to create an object of

Interface in Java

→ In English interface is a point where two systems meet and interact.



→ In java interface is group of related method with empty bodies.

Interfaces → implements

Abstract classes → extends

Interface supports multiple interface, class does not

Code Example

Interface Bicycle {

void apply brake (int decrement);

void speed up (int increment);

}

class Heron implements Bicycle {

int speed = 7;

void apply brake (int decrement) {

speed -= decrement;

}

void speed up (int increment) {

speed += increment;

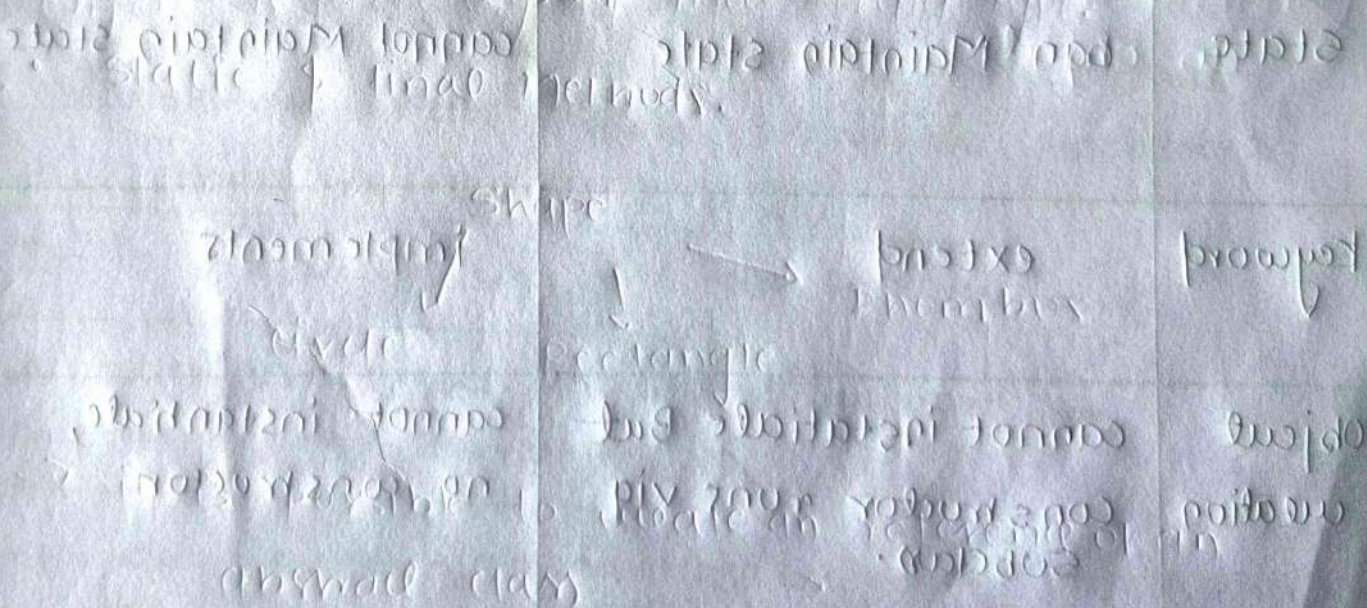
}

→ Important points to remember

- Interface is a contract which specifies what a class must do.
- A class that implements an interface must implement all the methods of interface.
- Before Java 8 interfaces would only have abstract methods (no bodies) since Java 8 they can also include default and static methods (with implementation) and since Java 9 private methods also.

→ Difference Between Abstract and interface.

→ Earlier interfaces had only abstract methods but after Java 8 they can have default, static and private methods, however, interfaces still cannot have constructors or instance variables, which is the key difference from abstract.



features	Abstract class	Interfaces
Purpose	Represent "IS-A" relationship with partial implementation	Represent contract and capability
Methods	can have abstract or concrete Method	can have abstract, default, static (java 8) private (java 9)
variable	can have instance variable	only public, static final ()
inheritance	A class can only extend only one abstract class	A class can implement multiple interfaces
Method access	can have access Modifier	Methods are public By default.
State	can Maintain state	cannot Maintain state
keyword	extend	implements
object creation	cannot instantiate But constructor runs via subclass.	cannot instantiate, no constructor

→ Abstract class vs Interface

→ we can't extend multiple abstract classes but we can implement multiple interfaces at a time

→ Interfaces are meant for dynamic Method dispatch and runtime polymorphism

Q) Is Multiple inheritance allowed in java?

Solⁿ → java does not support multiple inheritance through classes, meaning a class cannot extend two classes at the same point time. This is mainly to avoid ambiguity, especially the diamond problem.

However java does support multiple inheritance through interfaces, when a class can implement multiple interfaces.

Note → 1) An interface is a blueprint/contract that defines what a class should do rather than how should it do it

2) It is declared using the interface keyword.

3) A class uses the implements keyword to implement an interface

4) Interfaces support abstraction by hiding implementation details and exposing required behaviour

- 5) A class can implement Multiple interfaces which is java's way of supporting Multiple inheritance.
- 6) Methods declared in an Interface are public and abstract by default (unless they are default, static or private methods).
- 7) variable in an interface are public, static, and final by default, so they are constants.
- Interface achieves loose coupling
- 8) A class must provide implementations for the interface's abstract Methods, unless the class itself is declared abstract.
- 9) Since Java 8, interfaces can also contain default and static Methods with implementation.
- 10) you can use reference of interface but not an object

Default Method → interface can have static & default method

→ Default Method helps us to add New functionality to existing interface

→ feature was added in java 8 to ensure backward compatibility while updating interface

→ classes implementing the interface need not to implement the default method

→ Interfaces can also include private method for default method to use.

Inheritance in Interface

• Interface can extend another interface → can an interface extend to another interface

Ex. public interface Interface1 {

→ void meth1();

→ }

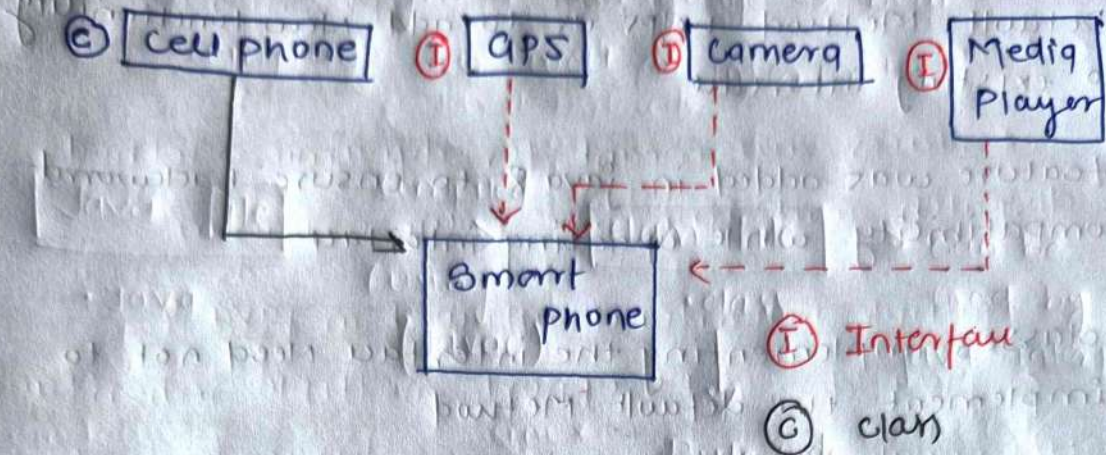
→ public interface Interface2 extends Interface1 {

→ void meth2();

→ }

Remember that interface cannot implement another interface only classes can do that.

> Polymorphism using Interface



→ `GPS g = new smart phone();` → can only use GPS methods

→ `Smart phone s = new smart phone();`

→ can use all methods

→ Implementing an interface forces method implementation

✓ Chapter II practice set completed completed.

Packages - 12.

Interpreter vs compiler

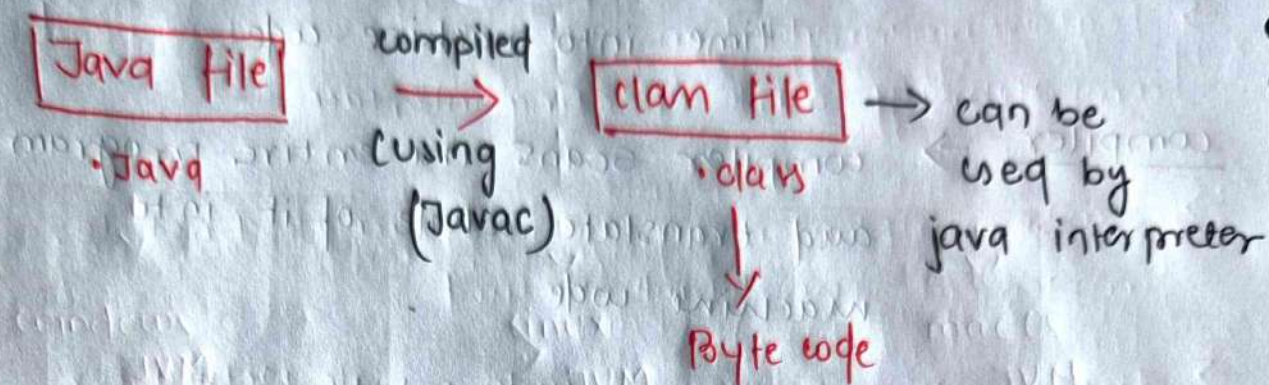
Interpreter → translates one statement at a time into machine code

compiler → compiler scans the entire program and translate whole of it into machine code

<u>Interpreter</u>	<u>compiler</u>
* one statement at a time * Interpreter Needed every time * partial execution if error * Easy for programmer	* Entire program at a time * Once compiled it is not Needed * No execution if an error occurs * usually not as easy as interpreter ones

→ Is Java compiled or interpreter?

Java is a hybrid language → both compiled as well as interpreter



→ WORKING OF JAVA ~~IMP.~~

JDK → Java development kit (complete toolkit required to develop Java application)

JDK contains →

JRE
JVM

Development tools

- Java C
- Java
- Java doc
- Other tools

→ JDK is used by developers to create compile, and run Java programs

• java → source code

Javac → Java compiler

JRE → Java Runtime Environment

→ JRE provides the environment required to run java environment.

JVM → Java Virtual Machine

→ The JVM is responsible for executing bytecode

→ Execution process

1) Main.java (source code) → Human understand

↓
compiled by (javac)

(javac Main.java)

↓ converts into

(Main.class) → (contains bytecode)

Bytecode is not normal CPU machine code

Why Bytecode?

→ imagine you compile java on windows Main.java

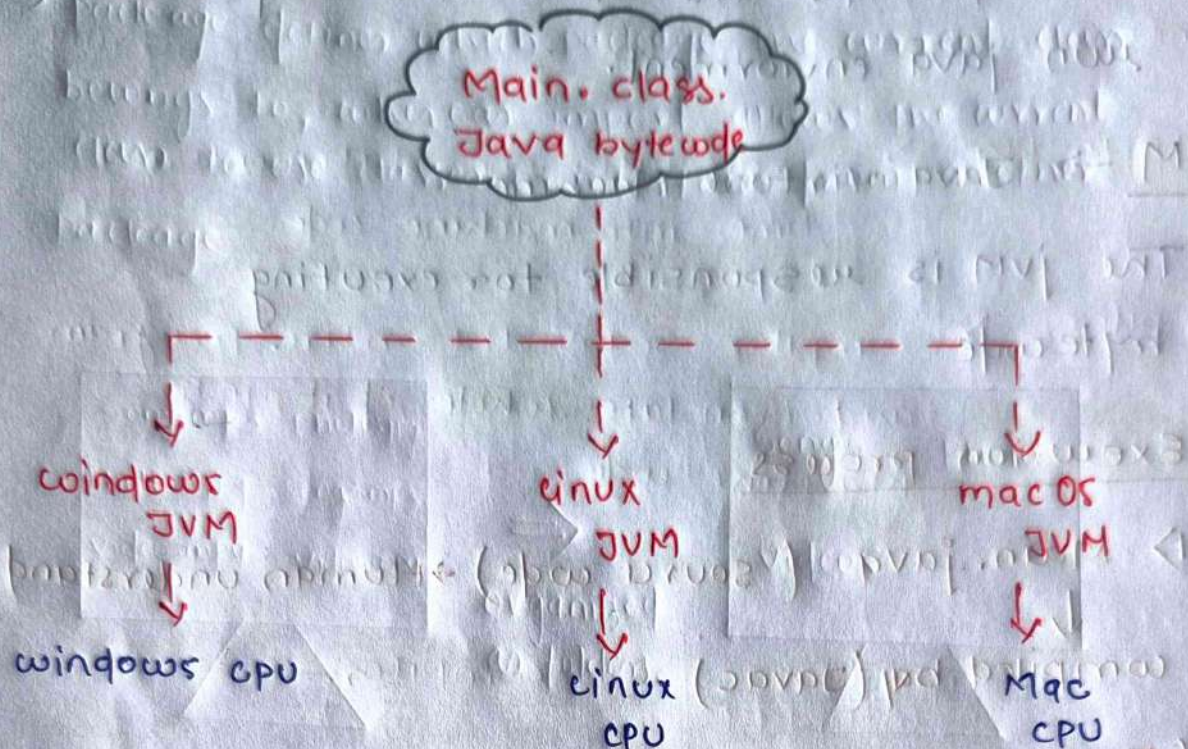
↓
Main.class

Now you copy Main.class to linux you don't normally need to recompile the java source

just because the OS changed Because the JVM

on each platforms understand the same Java bytecode

Diagrammatic Represⁿ



→ that's why java is said **(write once)** **run anywhere**

→ The JVM is platform-specific, while Java bytecode is designed to be platform-independent.

→ **JVM** → Runs Java bytecode

→ **JRE** → provides the runtime environment conceptually needed to run java program

→ **JDK** → provides the tools to develop java program along with runtime component.

Q) is java source code platform independent?

→ Java source code is compiled into platform independent bytecode, that bytecode can run on different OS using a platform specific JVM therefore, Java follows the principle of write once, Run Anywhere.

→ Executing a java program

Javac Kautik.java → compiled

java Harry.class → Interpreted

PACKAGES IN JAVA

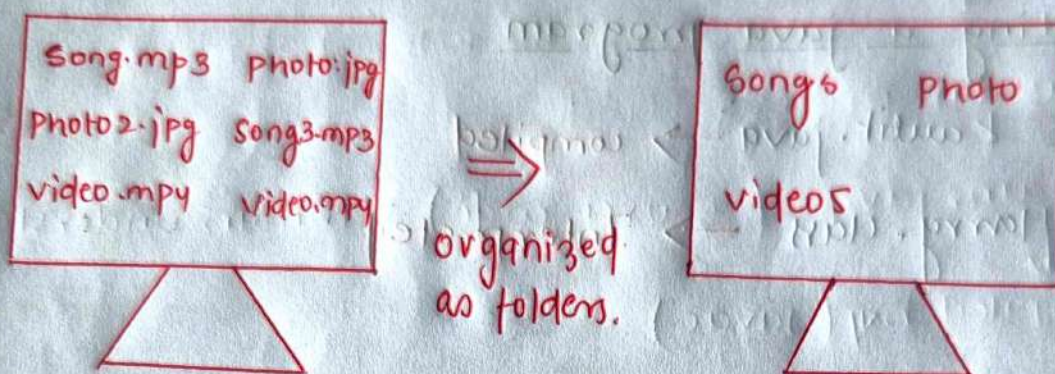
→ A package in java is a mechanism used to group and organize related classes, interface, and other components. it helps in better code organization, avoid naming conflict and provide access control.

why do we use package

we use package mainly for 3 reasons: to organize related classes, to avoid naming conflicts between classes with the same name, and to provide access control.

Q) What is the difference between package and import?

→ package defines which package the current class belongs to, whereas import allows the current class to use classes or interfaces from another package.



using a java package

→ `import java.lang.*` → import everything from java.lang

→ `import java.lang.String` → import String from java.lang

`S = new java.lang.String("Harry")` → use without importing

creating a package

`javac Harry.java` → creates Harry.class

`javac -d . Harry.java` → creates a package folder

↳ we can keep adding
classes to a package
with this.

we can also create inner packages by adding
"package .inner" as package name

folder

subfolder

These packages once created can be used by other
classes.

➤ A package in java is a mechanism used to
group and organize related classes, interface,
and other components. It helps in better
code organization, avoid naming conflict and
provide access control.

Why do we use package

we use package mainly for 3 reasons: To organize related
classes, to avoid naming conflicts between classes with
the same name, and to provide access control.